

Coding Cup 2019

Lösningar

A. Workout

Problem

Starting weight X

K changes: each change either +25 or -10% (new weight rounded down)

Can you reach a given target weight Z ?

Author: Johan

A. Workout

Solution

Easy subtask: simulate all possible choices of success or fail.

A. Workout

Solution

Medium subtask ($K \leq 10000$): simulate all possible choices of success or fail, but make sure you compress your intervals rather than keeping them as points in a set.

A. Workout

Solution

Assume that the sequence of changes is something like

++--+-+--+

What happens when swapping a +- to a -+?

If it was X before, it will change from

$$\text{floor25}((X + 25) * 0.9) = \text{floor25}(X * 0.9 + 22.5)$$

$$\text{to } \text{floor25}(X * 0.9) + 25 = \text{floor25}(X * 0.9 + 25)$$

Clearly, it will decrease by at most 25 after the swap.

A. Workout

Solution

Can it decrease by more than 25 in total?

No!

If you have two weights X and $X + 25$, a - operation will affect them as:

$$X \rightarrow \text{floor25}(X * 0.9)$$

$$X + 25 \rightarrow \text{floor25}((X + 25) * 0.9) = \text{floor25}(X * 0.9 + 22.5)$$

which can differ by at most 25.

A. Workout

Solution

How can we use this insight?

If we have a sequence $++++--\dots---$ and a sequence $--\dots---++++$ giving us weights X and Y , we can also form all weights between X and Y by swapping some pairs of $+-$ to $-+$ repeatedly.

This gives us a K^2 solution: for each number of $-$, generate the two above intervals and check if Z is in it.

This solves subtasks with $K \leq 1,000$ (and maybe $K \leq 10,000$ in well-optimized C++)

A. Workout

Solution

However, if we put more than $400 = -\log(10^{18}) / \log(0.9)$ - in the beginning, we get to 0 - then we already know the minimum of all intervals using more - operations. Thus, we can simply check if the target weight is at most $(K - 400) * 25$, otherwise we only need at most 400 - operations.

B. Typo Squatting

Problem

Given a set of strings, find for each string the number of other strings differing in exactly 1 position

Author: Johan & Simon

B. Typo Squatting

Solution

Easy solution: for each string, try replacing each character with a-z or 0-9, and put them all in a hash map. Then, for each string in the input, just check how many times you found it in the hash map.

Linear in the input, but too slow.

B. Typo Squatting

Solution

Better: for each string, try replacing each character with a-z or 0-9 **with a new character** ?, and put them all in a hash map. Then, for each string in the input, try replacing each character with a ? and see if you find it in the hash map

Loses a factor 2 because you need to perform twice as many lookups, but wins a factor 36 because you don't need to loop through the alphabet.

kodsport => ko?sport

kolsport => ko?sport

B. Typo Squatting

Solution

But don't you have to compute a lot of hashes of your new strings when replacing with ?

Yes, but this can be done in $O(1)$ per hash using polynomial hashes:

$$H(abcdef) = a * p^5 + b * p^4 + \dots + e * p + f$$

Now you can replace a character by removing e.g $b * p^4$ and adding $? * p^4$.

Precomputing all p^k makes this $O(1)$

C. Candy Packing

Problem

Given some kinds of candies of sizes $A_1, A_2, \dots, A_M$, split them into as many groups of K candies as possible, such that no group contains only 1 kind of candy, and no kind of candy is part of only a single group (beside kinds of size 1)

May take a candy from a group to create a new group of size 1

Author: Johan & Simon

C. Candy Packing

Solution

Actually quite easy.

C. Candy Packing

Solution

Actually quite easy.

Except that special case you missed.

C. Candy Packing

Solution

Actually quite easy.

Except that special case you missed.

And that second special case you missed.

C. Candy Packing

Solution

Actually quite easy.

Except that special case you missed.

And that second special case you missed.

Nah, your solution is just plain wrong :(

C. Candy Packing

Solution

General idea: we always need to throw away $\text{SUM}(A) \% K$ candies. Once this is done, we can always form groups with the remaining candy (for example, by making all of them unique new kinds).

In most cases, we only need to throw away a bunch of candies, and it's always possible to form groups with the remaining candy!

C. Candy Packing

Solution

An exception to this is if there is one huge kind, so that we cannot create groups not containing only that kind of candy.

Generally, this does not happen as long as

$$\text{MAX} * (K - 1) \leq (\text{SUM} - \text{MAX})$$

C. Candy Packing

Solution

if we remove Y of the biggest kind, we get that

$$(MAX - Y) * (K - 1) \leq (SUM - MAX + Y)$$

$\Leftrightarrow$

$$MAX - SUM * (K - 1) / K \leq Y$$

So we must remove at least $MAX - SUM * (K - 1) / K$ candies from the biggest group

C. Candy Packing

Solution

An exception to this is if we can only make a single box of candy -- then the smart greedy pairing the previous idea is based on doesn't work.

In this case, we start by taking all single candy kinds and all but one candy from the remaining kind -- this is OK!

Then, we might have to pick a few kinds to take their last candy from, turning all of them into single candies -- in this case, we start by picking the smaller groups until we are done

C. Candy Packing

Solution

But this assumes you have more than 1 kind. There is a special case:

If you have between $K + 1$ and $2K - 1$ candies (and would normally use the second solution), you should instead use the first solution.

D. Streaming Services

Problem

You want to watch TV every day, N days in a row.

Every day, either service 1 or service 2 or both have something worth seeing.

Subscriptions run for K days. How many subscriptions do you have to buy?

Author: Johan & Simon

D. Streaming Services

Solution

DP.

For $O(NK)$: “if we start subscription i at day j , and we have k days remaining on the other subscription, how many subscriptions do have to buy to cover the days $j..N$?”

Gives $2 \cdot N \cdot K$ DP states, and we can evaluate each state in $O(1)$ time with some pre-computation

D. Streaming Services

For $O(N \log K)$: in the previous DP, for fixed values of i, j , the result differs by at most 1

So we can instead do the DP: “if we buy subscription i on day j , what is the minimum number of subscriptions we need to buy to cover days $j..N$, and for that minimum number, how far back can we buy a subscription of the other kind?”

$2 \cdot N$ DP states

Can evaluate each one in $O(\log k)$ time using binary search

E. Mirror, Mirror...

Problem

Given a set of points, find the largest mirror symmetric subset

Author: David Avellan

E. Mirror, Mirror...

Solution

If $N = 1$, the answer is 1

Otherwise, there are either ≥ 2 points on the symmetry line, or ≥ 2 points mirrored across it

Find all possible symmetry lines by looping over all pairs of points

- How many points are mirrored across a candidate symmetry line?
- How many points lie on a candidate symmetry line?

E. Mirror, Mirror...

How many points are mirrored across a candidate symmetry line?

When looping over pairs of points, we can mark their midpoint normal line as containing two more points

Represent lines as e.g. intersection with $y = 0$ and slope (as doubles or rationals)

E. Mirror, Mirror...

How many points lie on a candidate symmetry line?

$O(\log n)$ solution: keep a set of points sorted by the projection onto the orthogonal line, then binary search to find the points exactly on the right line

Consider symmetry lines in order of angle, and update this set incrementally

E. Mirror, Mirror...

How many points lie on a candidate symmetry line?

Maybe also $O(\log n)$ solution (???): for every point, sort all other points by distance

Two points at the same distance with have this point on its midpoint normal

Worst case: grid filled with points?

E. Mirror, Mirror...

Sets may be too slow, hash maps are better

Or sorting + binary search/sweep

Can also win some performance by breaking early

F. Mutexes

Problem

Given a function call graph, with statements of the kinds:

- `assert(x == 0); x = 1;`
- `assert(x == 1); x = 0;`
- `assert(x == 1);`

Do any of the asserts fail?

Author: Johan

F. Mutexes

Solution

Focus on one variable at a time

Do DP over “if this function is called with $x == \langle X \rangle$, does it fail, and if not, what is the output state of x ?”

To find the first error: simulate the program, only recurse into functions with errors

Can speed this up using bitsets